Phase 2 Design Rationale

Adapting Hyperbolic U-Net (MIDL 2026) for Retinal Image Denoising on RFMiD

Author	Ayoub Hidara
Date	May 22, 2026
Phase	Phase 2 — Architecture design and implementation
Scope	Justification of every architectural, loss, data, training, and hardware choice for the Eucli
Status	Design locked; implementation pending execution on Kaggle

Purpose of this document. A design rationale, written before implementation, that documents and defends every non-trivial decision shaping the Phase 2 baseline. It is structured so that any choice can be challenged ("*Why widths 32/64/128/256/512?*", "*Why L1 rather than MSE?*", "*Why predict noise rather than the clean image?*") and answered with a principled reference to either the literature, a measurable constraint of our compute environment, or an explicit trade-off accepted for stated reasons. The intent is that by reading this document a thesis committee can audit the reasoning behind the model *independently of* whether the empirical results are good — the methodology stands on its own.

1. Executive summary

Phase 2 of this project builds a U-Net-based denoiser for the RFMiD retinal fundus dataset. Rather than implement a U-Net from scratch, we adopt the published reference implementation of Mishra et al. (MIDL 2026) — *Hyperbolic U-Net for Robust Medical Image Segmentation* — and surgically adapt it for our regression task. The reference ships both Euclidean and Hyperbolic variants of the same skeleton, which is precisely what we need: it lets Phase 2 (Euclidean baseline) and Phase 3 (hyperbolic counterpart) share an identical encoder, decoder, skip-connection topology, and data pipeline, isolating geometry as the sole independent variable between the two runs.

The principal design decisions and their drivers are summarised below. Each is expanded with full justification in the sections that follow.

Decision	Choice	Driver
Base codebase	swastishreya/Hyperbolic-U-Net (MIDL 2026)	Official reference; provides matched Euclidean + Hyperbol
Backbone	U-Net, 4-level encoder/decoder, skips	Standard for dense prediction; used as-is from upstream.
Channel widths	32 / 64 / 128 / 256 / 512	Half of classic; fits Kaggle T4 16 GB at 256² × batch 16; suffic
Normalization	GroupNorm (or no norm)	BatchNorm interferes with the low-magnitude noise signal we
Output head	1×1 Conv → 3 channels, linear	Replaces upstream segmentation softmax head.
Forward semantics	Predict residual noise n̂; return y − n̂	DnCNN-style residual learning; well-documented ~0.5–1.0 dB
Loss	L1 (mean absolute error)	Sharper outputs than MSE on denoising/restoration tasks (Zha
Patch size	256 × 256 random crops	Standard in image-restoration literature; sufficient receptive f
Augmentations	Flips + 90° rotations only	Noise statistics are direction-invariant under flips/90° rotation
Noise injection	Poisson-Gaussian, on-the-fly, level sampled uniformly	Three levels from Phase 1's noise_config.json (light/medium/h
Optimizer	AdamW	Decoupled weight decay; stable default for dense-prediction C
LR schedule	Cosine annealing	Loshchilov & Hutter SGDR; standard, no-tuning schedule that
Precision	Mixed-precision (AMP, fp16 forward / fp32 master)	≈ 2× throughput and ≈ ½ memory on T4 Tensor Cores at neg
Metrics	PSNR + SSIM per noise level	Standard for image restoration; per-level breakdown gives a r

2. Strategic framing: adapt, do not build from scratch

The first design decision is methodological: do we implement a denoising U-Net from scratch, or do we fork a published, peer-reviewed reference and adapt it? We chose the latter, for four reasons.

2.1 Credibility with a non-specialist supervisor

The supervisor of this internship is not a specialist in hyperbolic deep learning. From her perspective, a result obtained by adapting an established reference is intrinsically more trustworthy than one obtained from a hand-rolled implementation, however correct the latter might be. Bugs in a from-scratch U-Net are *our* responsibility to find; the published reference has been reviewed by the MIDL programme committee, executed by its authors, and the code is publicly auditable. Anchoring on such a reference shifts the burden of proof in our favour.

2.2 Apples-to-apples Euclidean vs. hyperbolic comparison

The Phase 3 deliverable is a comparison between a Euclidean baseline and a hyperbolic variant. For that comparison to be scientifically meaningful, the two networks must differ *only* in the hyperbolic operations themselves — same depth, same widths, same skip topology, same data pipeline, same optimiser, same evaluation protocol. Two from-scratch implementations would inevitably drift apart in dozens of subtle ways (initialization, padding mode, the exact placement of normalization, weight-decay scope). The Mishra et al. codebase enforces parity by construction: a single source tree, a single `--model` flag, identical hyper-parameters. This is precisely the experimental hygiene the comparison demands.

2.3 The reference's motivation matches our objective

Mishra et al. evaluate their Hyperbolic U-Net specifically as a *noise-robust* alternative to standard U-Net. Their experimental protocol injects Gaussian, Speckle, Poisson, and Rician noise into the test images and measures the segmentation degradation. Their headline claim is that the hyperbolic variant is more robust to noise than the Euclidean variant. This is exactly the property we want to exploit: where they used noise robustness as a stress-test for segmentation, we promote noise removal to the task itself. The architecture's bias toward noise-robustness is therefore not just a lucky side-effect but the documented design intent of the reference.

2.4 Phase 2 and Phase 3 collapse into a single codebase

The original project plan separates Phase 2 (Euclidean baseline) and Phase 3 (hyperbolic variant) into two distinct engineering efforts. By adopting Mishra et al.'s codebase, both variants share a single repository and a single training loop. Phase 3 reduces, in software terms, to a single flag change. This frees roughly two weeks of nominal Phase 3 engineering time, which can be reinvested in evaluation, ablation, and the writing of the final report.

2.5 What we accept by this choice

Adopting an external codebase is not free. We accept (a) a dependency on the upstream library `hypll` (Hyperbolic Learning Library, van Spengler et al.), which constrains our PyTorch version and forbids certain low-level customizations; (b) the upstream's design decisions about how to implement Möbius operations and Riemannian optimization, which we inherit without independent re-derivation; (c) the risk that bugs in the reference will surface in our results. These risks are mitigated in Section 13 (Risks).

3. The choice of base paper

Several published U-Net variants could in principle have served as the base. The selection criteria, applied in priority order, were:

1. **Direct hyperbolic implementation available.** A repository containing a working hyperbolic U-Net, not merely a paper describing one.
2. **Matched Euclidean baseline in the same code tree.** So that the Euclidean/hyperbolic comparison is reproducible by flipping a single flag.
3. **Maintained and credible library dependencies.** The hyperbolic layers should rely on a well-maintained library (hypll, geoopt), not on research-grade code that may have been abandoned.
4. **Validated on medical imaging.** Ideally including a fundus or retinal dataset, so the input distribution is at least adjacent to RFMiD.
5. **Noise robustness as a stated design goal.** Aligns the reference's optimization with our denoising objective.

The Mishra et al. (MIDL 2026) paper satisfies all five criteria. The candidate evaluation is summarised below.

Candidate	Criteria met	Verdict
Mishra et al. 2026 (Hyperbolic U-Net)	All five	Selected. PyTorch, dual Eucl/Hyp variants, hypll, REFUGE2 fundus datase
Atigh et al. 2022 (Hyperbolic Image Segmentation, CVPR)	Partial	Hyperbolic gains over Euclidean backbone — not a fully hyperbolic U-Net
Bdeir et al. 2024 (HCNN)	Partial	Fully hyperbolic convolutions (Lorentz model). Important for a possible st
EyeQ_Enhancement (Fu et al., TMI)	Partial	Excellent retinal-specific preprocessing and fundus-circle masking, but E
From-scratch implementation	—	Rejected on the grounds discussed in Section 2.

3.1 What the paper provides — concretely

Inspection of the Mishra et al. repository confirms the following components are production-ready and directly importable:

- **Encoder, bottleneck, and decoder blocks** in both Euclidean and hyperbolic variants, with a configurable depth and base channel count (`--depth`, `--init_feats`).
- **Skip connections** in the standard U-Net pattern (concatenation in the decoder).
- **Riemannian optimization** via Riemannian Adam (for the hyperbolic model), with curvature itself optionally trainable (`--trainable`).
- **A clean training loop** with mixed-precision support (`--amp`), validation cadence (`--validation`), and W&B; logging.
- **Multi-dataset dataloaders** for ISIC, REFUGE2, CVC-ColonDB, BUSI, KVASIR — proof that the data interface is generic enough to accept a new dataset class easily.

3.2 What the paper does not provide

The reference is a segmentation framework. Out of the box it does not provide: (a) a regression-mode output head; (b) restoration losses (L1, Charbonnier, perceptual); (c) restoration metrics (PSNR, SSIM); (d) on-the-fly noise injection; (e) a paired-image

dataloader for denoising. Each of these is built or wired in this Phase, and the work is enumerated as the *adaptation deltas* in Section 11.

3.3 Reading the citation

Mishra, S. S., van Spengler, M., Berkhout, E., & Mettes, P. (2026). *Hyperbolic U-Net for Robust Medical Image Segmentation.* Medical Imaging with Deep Learning (MIDL). OpenReview: NxKaeTNMxR. Repository: github.com/swastishreya/Hyperbolic-U-Net.

4. Architectural decisions

4.1 The U-Net family — why this shape at all

U-Net (Ronneberger, Fischer & Brox, MICCAI 2015) is the canonical architecture for dense pixel-level prediction tasks. It has three properties that map cleanly onto the denoising problem:

- **Encoder–decoder topology.** The encoder progressively downsamples to build a multi-scale feature hierarchy; the decoder upsamples back to the original resolution. For denoising, this matters because noise has structure at multiple spatial frequencies — fine-grain shot noise, JPEG-block artefacts at 8-pixel scale, illumination drift at hundreds of pixels — and the multi-scale hierarchy lets a single network handle all of them.
- **Skip connections.** Direct concatenations from each encoder level into the matching decoder level. This preserves high-frequency detail that would otherwise be lost in the downsampling. In a denoising context the skips are not optional: without them, the decoder would have to synthesise edges and fine vasculature from a compressed bottleneck representation, which inevitably blurs the output.
- **Fully convolutional.** No fully-connected layers; the network operates patch-wise and tile-wise. This is what lets us train on 256×256 crops and inference on full 2144×1424 images without retraining.

Alternatives considered and rejected for the baseline: **Restormer / SwinIR** (transformer-based restoration networks) — state-of-the-art on natural images, but heavier and less well-supported with hyperbolic counterparts; **DnCNN** (a plain residual CNN, no encoder/decoder) — lighter but lacks the multi-scale structure that makes U-Net a natural Phase-3 host for hyperbolic bottleneck operations. The Phase 1 literature review explicitly motivates U-Net as the chosen backbone, and the supervisor's reference to "segmentation" in the project brief was a hint to use a segmentation-style backbone for the denoising task.

4.2 Depth — four downsampling levels

We use a 4-level encoder/decoder (5 total resolutions including the bottleneck): 256 → 128 → 64 → 32 → 16. This matches the upstream reference and the original Ronneberger et al. design. The rationale is a receptive-field argument.

At 256² input with two 3×3 convolutions per encoder block and 2× downsampling between blocks, the bottleneck features have a theoretical receptive field of approximately 140 × 140 input pixels per neuron. This comfortably covers a single vessel branching junction, a small lesion, or several JPEG blocks. Going deeper (5 levels) would give an irrelevantly large receptive field for the patch we're working on; going shallower (3 levels) would leave the bottleneck without enough context to model slow brightness gradients that span larger regions.

4.3 Channel widths — 32 / 64 / 128 / 256 / 512

The classic U-Net of Ronneberger et al. uses channel widths 64 / 128 / 256 / 512 / 1024. We use exactly half: **32 / 64 / 128 / 256 / 512**. The choice is driven by three convergent constraints.

Constraint A — Kaggle T4 memory budget.

The free Kaggle GPU is an NVIDIA T4 with 16 GB VRAM. The activation memory of a U-Net is dominated by the first encoder level (largest spatial resolution × full channel count). For batch size B and the first encoder level, the forward activations alone consume:

> **mem ≈ B × C_1 × H × W × 4 bytes × ≈ 3** (forward + saved-for-backward + Adam state)

With B = 16, C_1 = 32, H = W = 256 this gives ≈ 400 MB *just for the first encoder level*. Doubling to C_1 = 64 would push the same line to ≈ 800 MB. Add the deeper levels, the parameters themselves, the gradient buffers, the optimizer state, the data loader's pinned memory, and the cuDNN workspace, and the 64-width version becomes uncomfortable at batch size 16 on a T4. Halving the widths halves the activations and keeps comfortable headroom for AMP, autograd buffers, and the dataloader.

Constraint B — denoising is structurally low-capacity.

Denoising is fundamentally different from classification. The output space is isomorphic to the input space (an RGB image with the same spatial dimensions and intensity range), and the target function is approximately identity-plus-small-correction. Empirically, much smaller networks suffice for denoising than for segmentation or recognition. DnCNN (Zhang et al. 2017), the foundational deep denoising network, achieves state-of-the-art Gaussian-noise PSNRs with only ~0.6 M parameters — orders of magnitude below the classic U-Net's ~31 M. By halving the widths we still budget ~7 M parameters, which is comfortable headroom over DnCNN's proven envelope.

Constraint C — fair comparison with the hyperbolic variant.

Hyperbolic layers are more memory- and compute-intensive than their Euclidean counterparts (exp/log maps, projection clamping, Riemannian moments). If the Euclidean baseline is already at the memory limit, the hyperbolic counterpart will OOM at the same batch size, forcing us to reduce the batch and re-tune the learning rate for Phase 3. Choosing widths that leave the Euclidean baseline with ~30% memory headroom keeps the Phase 3 transition frictionless.

4.4 Normalization — GroupNorm or none, not BatchNorm

The upstream reference uses BatchNorm. For our denoising adaptation we switch this to GroupNorm (or, equivalently for the Euclidean baseline, no normalization at all).

The argument against BatchNorm for denoising is well-established in the restoration literature. BatchNorm normalises each channel by the batch statistics, effectively removing constant brightness and contrast offsets from the feature maps. In a classification or segmentation context this is desirable: the network should not care whether the lighting is bright or dim. In a denoising context it is harmful: noise itself is a small, low-energy perturbation of the signal, and BatchNorm's running statistics can swamp it. EDSR (Lim et al. 2017) — the network that won NTIRE 2017 super-resolution — explicitly removes all batch normalization layers as a primary design innovation, reporting *both* better PSNR and reduced memory consumption. Subsequent restoration work (EDVR, MIRNet, NAFNet) has overwhelmingly followed this practice.

GroupNorm (Wu & He, ECCV 2018) is the safe middle ground. It normalises within groups of channels for each *individual* sample, ignoring batch statistics entirely. It

therefore retains the training-stability benefit of normalization without coupling samples or destroying the noise signal. It is also batch-size-invariant, which matters because our T4 batch size (≈ 16 at 256^2) is modest.

4.5 Activation function — ReLU, kept from upstream

We retain ReLU. Alternatives (LeakyReLU, GELU, SiLU/Swish) have marginal effect on denoising performance in the literature and would constitute a gratuitous deviation from the upstream reference. The fewer differences between our codebase and Mishra et al., the easier it is to attribute any observed performance change to the modifications we *did* make.

5. Output head and residual learning

This is the single most important architectural decision distinguishing our adaptation from the upstream segmentation model, and we devote a full section to it because it addresses one of the questions a thesis committee is most likely to ask: *why predict the noise, rather than directly reconstructing the clean image?*

5.1 The two possible formulations

There are two natural ways to formulate the denoising forward pass.

Formulation A — direct reconstruction.

The network takes the noisy image y and outputs a prediction $\hat{x}$ of the clean image directly:

$$\mathbf{\hat{x} = f(y)}, \text{ loss} = \|\hat{x} - x\|$$

This is conceptually the simplest, and it is what a freshly-converted segmentation network would do without modification (the network's output is a pixel-level RGB image, taken as the answer).

Formulation B — residual learning.

The network instead predicts the noise $\hat{n}$; the clean estimate is recovered by subtraction:

$$\mathbf{\hat{n} = f(y)}, \mathbf{\hat{x} = y - \hat{n}}, \text{ loss} = \|\hat{x} - x\| = \|(y - \hat{n}) - x\| = \|\hat{n} - (y - x)\|$$

The last equality reveals the geometry: the network is fitting the residual $r = y - x$, i.e. the actual noise. Crucially, the loss can be written equivalently as $\|\hat{n} - r\|$: **this is a supervised regression to the ground-truth noise**.

5.2 Why residual learning wins

Argument 1 — the network learns a smaller, easier function. The clean image x is a complex, structured, high-dynamic-range signal: vasculature, optic disc, lesions, background gradient. The noise r has much smaller magnitude (peak $\sigma \approx 18$ at heavy noise vs. peak intensity 255), narrower variance, and is approximately stationary. From a function-approximation standpoint, fitting r is therefore a substantially simpler problem than fitting x, especially when the input y already *contains* the answer to most of x — the network only needs to add a small correction. Empirically this is consistently worth 0.3–1.0 dB in PSNR across denoising benchmarks (Zhang et al. 2017, He et al. 2016 on the general residual-learning principle).

Argument 2 — better gradient flow at convergence. When the network is well-trained, $\hat{x} \approx x$ means that in Formulation A the output layer is producing values close to the input — the network must *reconstruct* the input. The gradients on each output pixel are small in magnitude but the output values are large, leading to a low signal-to-noise ratio in the gradient updates and slow late-stage convergence. In Formulation B the output values (the noise) are small in magnitude and the gradients act on those small values directly — the network is always working in the regime where its output is non-trivial.

Argument 3 — implicit identity skip. Residual learning effectively adds a skip connection from the input to the output of the entire network. This is the same insight as

ResNet (He et al. 2016): it makes the optimisation landscape dramatically easier because the gradient has a direct path from loss back to input, bypassing the depth of the network and avoiding the vanishing-gradient problem on the residual path. In a U-Net this is on top of the existing within-network skip connections, and the two compose.

Argument 4 — when the model is wrong, it fails gracefully. If the network's prediction n̂ is identically zero (the trivial solution), Formulation A returns a black image (catastrophic), but Formulation B returns y itself — a noisy version of the right image. The worst-case output of residual learning is the input, which is far less harmful than the worst-case output of direct reconstruction.

5.3 The concrete head architecture

After the final decoder block, the output head is a single 1×1 convolution:

```
Input: decoder feature map (B, 32, 256, 256)
Op: Conv2d(32 → 3, kernel=1, bias=True)
Output: noise prediction nˆ (B, 3, 256, 256), linear (no activation)

Forward returns: ŷ = clip(y − nˆ, 0, 1) # denoised image in [0,1]
```

Two implementation notes. First, **no output activation**: a sigmoid or tanh would constrain the noise prediction to a bounded range, which we do not want — the noise can in principle take any value, and clamping is applied at the level of the recovered image, not the noise itself. Second, the final clip to [0, 1] only matters at inference and validation; during training the loss is computed before clipping, so the gradient is not interrupted.

5.4 Cost

Replacing the segmentation head with a 3-channel regression head is a roughly 100-line change in the model file. The forward-pass wrapper that performs the y − n̂ subtraction is another 5 lines. No re-engineering of encoder, decoder, or skip-connection code is required. The change is therefore proportionally small relative to its impact on training behaviour.

6. Loss function

6.1 Candidates considered

Candidate	Strength	Cost / weakness
MSE (L2)	Smooth, matches PSNR exactly	Over-penalizes large residuals; produces blurry outputs in th
L1 (MAE)	Sharper outputs, robust to outliers	Discontinuous gradient at zero (cosmetic; in practice harmle
Charbonnier	Smooth approximation of L1: $\sqrt{(\hat{y}-y)^2+\varepsilon^2}$	Differentiable everywhere; marginal gain over L1; one extra
L1 + α·(1 − SSIM)	Best perceptual quality	Requires an SSIM module; adds a tunable weight; second-or
Perceptual (VGG-feature L2)	Aligns with human perception	Requires a fixed pre-trained VGG; couples the model to Imag

6.2 Why L1 was chosen

The decisive argument is the Zhao et al. paper *Loss Functions for Image Restoration with Neural Networks* (IEEE TCI 2017), which systematically compared MSE, L1, SSIM, and combinations across denoising and super-resolution benchmarks. Their headline finding is that **L1 outperforms MSE on every metric they measured, including PSNR** — counter-intuitive because MSE is mathematically equivalent to maximizing PSNR, but L1 converges to a sharper solution that the eventual PSNR also rewards.

The intuition: MSE's quadratic penalty creates a strong preference for averaging across plausible reconstructions whenever the model is uncertain. In the presence of noise this uncertainty is everywhere, and the result is a low-pass filtered output. L1's linear penalty is indifferent to whether a given error is small or moderately large, so the optimiser is content to commit to one reconstruction rather than blend two. The consequence is sharper edges and finer vessel detail — exactly what we want in a retinal denoiser.

L1 is also the loss used by EDSR (Lim et al. 2017), CycleGAN (Zhu et al. 2017), and Pix2Pix (Isola et al. 2017) — all foundational image-to-image works, all on the L1 side of the choice. We follow the same default.

6.3 Why we defer Charbonnier and SSIM-augmented losses

Charbonnier (a smoothed L1) typically yields 0.05–0.1 dB PSNR over plain L1 in image restoration. The gain is real but not large enough to justify the additional hyperparameter (ε) in a first baseline. We have the option to switch later if needed.

An L1 + SSIM combination is the current best-in-class for perceptual quality and would be the natural next step for an ablation. We do not include it in the baseline because (a) the SSIM module adds engineering complexity and a tunable weight, (b) the result becomes harder to interpret (is the network learning denoising, or is it learning to game the perceptual loss?), and (c) the baseline's purpose is to be a clean, defensible reference point against which Phase 3's hyperbolic variant is compared — adding extra losses now would muddy that comparison.

6.4 Loss is computed in the image domain, not the noise domain

Mathematically, $\|\hat{n} - r\|$ and $\|\hat{x} - x\|$ are equivalent objectives (Section 5.1). We compute the loss on the image-domain prediction $\hat{x} = y - \hat{n}$ rather than directly on $\hat{n}$, for two

reasons. First, it is the more natural way to write SSIM later (SSIM is defined on images, not noise residuals). Second, when we eventually add evaluation logging, it lets us reuse the same forward-pass tensors for both the loss and the metrics. Either choice produces identical training gradients in the L1 case.

7. Data pipeline

7.1 Patch size — 256 × 256 random crops

Training is performed on random 256 × 256 crops sampled per image per iteration. The choice of patch size involves three trade-offs.

Receptive field. A 4-level U-Net at 256^2 has a theoretical receptive field of ≈ 140 input pixels (Section 4.2), comfortably below the patch size. At smaller patches (e.g. 128^2) the receptive field saturates the patch, meaning the network sees the patch boundary in nearly every output pixel — borderline artifacts. At larger patches (e.g. 512^2) the receptive field is small relative to the patch, so most pixels still only see a local neighborhood, and the extra context is wasted while memory cost grows quadratically. 256^2 is the standard restoration-literature compromise (DnCNN, EDSR, NAFNet all use 48–192^2 for super-resolution, 64–256^2 for denoising).

Memory cost. Activation memory scales as $B \times C \times H \times W$. Halving H × W ($128^2$ vs 256^2) lets us roughly quadruple the batch size — useful for very small models, but for our U-Net with 32-wide first level we have no need; 256^2 × batch 16 already comfortably fits the T4. At 512^2 we would have to drop to batch ~4, which hurts BatchNorm replacements (though we use GroupNorm, so this is mostly a gradient-noise concern).

Data diversity. Random cropping turns each ~3000-pixel-wide RFMiD image into many non-overlapping 256^2 patches. With ≈ 8 patches per image × ~2000 training images per epoch × three noise levels sampled randomly, the effective number of unique training examples per epoch is in the tens of thousands. This is more than sufficient for the ~7 M parameter model not to overfit.

7.2 Why patches, not full-resolution training

RFMiD images are roughly 2144 × 1424 pixels. Training a U-Net at that resolution on a T4 is infeasible — the first encoder level alone would consume ≈ 16 × 32 × 2144 × 1424 × 4 × 3 bytes ≈ 18 GB, well past the 16 GB budget — and would gain us nothing in receptive field that 256^2 crops do not already provide. At inference time we will tile the full image into overlapping 256^2 patches, denoise each, and stitch back with a Hann window for smooth seams; this is standard practice in image restoration (see Restormer's tiling code or BasicSR's TiledTest helper).

7.3 Augmentations — flips and 90° rotations only

Allowed augmentations:

- Horizontal flip (probability 0.5)
- Vertical flip (probability 0.5)
- 90° rotation, randomly chosen from {0°, 90°, 180°, 270°}

These are the **dihedral group** augmentations: the 8 symmetries of the square. They preserve all properties of the noise distribution: Poisson-Gaussian shot+read noise is spatially independent (so direction does not matter) and the augmentations are permutations of pixels (so the noise distribution at each pixel is unchanged).

What we forbid, and why.

- **Colour jitter (brightness, contrast, saturation, hue).** These shift or rescale pixel values, which directly changes the Poisson noise variance at every pixel (since var = α · μ + σ², the variance depends on the mean). The clean image our network is being asked to recover would no longer match the noise statistics injected into the input — we would be training the network on a different denoising problem than the one we will evaluate on.
- **Geometric warps (affine, perspective, elastic).** Sub-pixel resampling interpolates the noisy pixels and effectively low-pass-filters the noise, again destroying the noise statistics we are training the model to invert.
- **Gaussian blur, sharpening, gamma.** Same reason — they corrupt the signal we are trying to learn.
- **MixUp / CutMix.** They violate the (noisy, clean) pairing.

Note on order of operations. The augmentation is applied to the *clean* image, and noise is injected *after*. This ensures the noise is fresh, statistically independent of anything that came before, and matches the noise statistics that test images will be evaluated under.

7.4 On-the-fly noise injection

The dataloader stores only the clean RFMiD images. At each `__getitem__` call, the dataloader:

1. Loads the clean image, converts to RGB float in [0, 1].
2. Applies a random crop to 256 × 256.
3. Applies the dihedral augmentation (one of the 8 symmetries, chosen uniformly).
4. Samples a noise level uniformly from {light, medium, heavy} (parameters from Phase 1's noise_config.json).
5. Injects Poisson-Gaussian noise: $y = x + \sqrt{(\alpha \cdot x)} \cdot N(0,1) + \sigma \cdot N(0,1)$, with x scaled appropriately (the formula in Phase 1 is on 0–255; here on 0–1 we use $\alpha' = \alpha/255$ to keep the same physical noise magnitude).
6. Returns the pair (noisy, clean).

Why on-the-fly, not pre-computed. Pre-computing one fixed noisy version per training image would mean the network sees the same realisation of the noise at every epoch — it could in principle memorise the realisation rather than learn the denoising operator. On-the-fly injection draws a fresh noise sample every iteration, so the network sees an essentially infinite number of unique (noisy, clean) pairs, which is the closest we can get to a non-overfitting regression target.

Why mixed noise levels rather than a fixed level. Training on all three levels simultaneously yields a single *blind denoiser* that performs reasonably across the range. Evaluating it per-level then gives a robustness curve (PSNR as a function of input noise) — a more informative result than three separately trained, level-specific models. It also matches Mishra et al.'s evaluation protocol, which tests the same model against multiple noise types.

7.5 Train / validation / test split

RFMiD ships with three pre-defined splits: Training_Set (1920 images), Evaluation_Set (640), Test_Set (640). We honour these splits exactly — using a different split would compromise comparability with any prior RFMiD work and would risk patient-level data leakage between train and test (the official splits are constructed to avoid this).

8. Training recipe

8.1 Optimizer — AdamW

AdamW (Loshchilov & Hutter, ICLR 2019) is Adam with decoupled weight decay. The decoupling matters: in vanilla Adam, weight decay is folded into the gradient and therefore rescaled by Adam's adaptive learning rate, which interacts badly with the running gradient statistics. AdamW applies weight decay directly to the parameters as a separate term, which is mathematically cleaner and empirically yields better generalisation across vision tasks.

Default hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, ε = 1e-8, weight decay = 1e-4. These are the standard values used throughout the restoration literature; we have no reason to deviate.

Initial learning rate: 1×10^{-4}. The Mishra et al. reference uses 1e-3 for segmentation; we drop one order of magnitude because (a) regression targets have much smaller per-pixel range than softmax logits, so the loss landscape is gentler and high LRs overshoot; (b) the residual-learning formulation already gives the network an easy starting point (predicting zero noise gets close to the input, which is close to the answer), so we want fine-grained tuning rather than aggressive steps. 1e-4 is the standard starting LR for image restoration with Adam (DnCNN, EDSR, NAFNet).

For Phase 3 (hyperbolic): we will switch to Riemannian Adam on the hyperbolic parameters (curvature, Möbius weights) and keep AdamW on the Euclidean parameters (skip connections, convolutional weights of Euclidean layers if any remain). hypll wires this up automatically; we just specify the optimiser type.

8.2 Learning rate schedule — cosine annealing

Cosine annealing (Loshchilov & Hutter, ICLR 2017, *SGDR*) varies the learning rate from its initial value down to (near) zero over the course of training, following a half-cosine curve:

$$\mathrm{lr}(t) = \mathrm{lr}_{\min} + \tfrac{1}{2}\,(\mathrm{lr}_{\max} - \mathrm{lr}_{\min}) \cdot (1 + \cos(\pi \cdot t / T))$$

where T is the total training step count. The schedule has three desirable properties: (a) no step boundaries to tune — unlike step decay, which forces a choice of when to drop the LR and by how much; (b) the late-training low LR allows the model to settle into a flat minimum, which generalises better; (c) it is the standard schedule in the recent restoration literature (NAFNet, Restormer, SwinIR), so our results will be comparable to published numbers without a schedule-specific footnote.

We schedule per-iteration, not per-epoch, so the cosine curve is smooth across the entire training run. $\mathrm{lr}_{\min}$ = 1e-7 (effectively zero); $\mathrm{lr}_{\max}$ = 1e-4.

8.3 Mixed-precision training (AMP)

We enable PyTorch's automatic mixed-precision (`torch.cuda.amp.autocast` + `GradScaler`). The forward pass runs in fp16 where safe and fp32 where required (reductions, the loss); weights are stored in fp32 master copies. On a T4 with Tensor Cores this yields ≈ 1.6–2.0× wall-clock throughput and roughly halves activation memory at negligible cost in PSNR (typically below the noise floor of run-to-run variance).

Important caveat for Phase 3. Hyperbolic operations (arctanh in particular) are numerically delicate and prone to overflow in fp16 as $\|y\|$ approaches 1. The standard mitigation is to clamp $\|y\|$ to $\approx 1 - 10^{-5}$ before calling arctanh, and to force fp32 around the exp/log/Möbius operations themselves (autocast supports per-op casting). The hypll library already handles this internally; we will verify it in the Phase 3 hyperbolic notebook.

8.4 Batch size — 16, with gradient accumulation as a fallback

Batch size 16 at 256^2 fits comfortably on a T4 in our configuration (Section 9 calculates the memory budget). If a Phase 3 hyperbolic run exceeds memory at this batch, we will halve to 8 and apply 2-step gradient accumulation, which is mathematically equivalent to batch 16 for optimisation purposes. The AdamW + cosine recipe is robust to batch sizes in the 8–32 range; we do not need to retune for either endpoint.

8.5 Number of epochs

Initial budget: 100 epochs (≈ 12 hours of T4 time). We expect convergence (validation PSNR plateauing) well before that based on DnCNN and EDSR training curves on comparable datasets — typically 60–80 epochs. The cosine schedule is parameterised to the full 100, so early stopping would forgo the final LR-annealing benefit; we let it run end-to-end and use the best-validation checkpoint.

9. Hardware constraints and the memory budget

All training is performed on Kaggle's free GPU resource (no local GPU available). At the time of writing, Kaggle allocates either an NVIDIA T4 (16 GB, 2,560 CUDA cores, mixed-precision Tensor Cores) or, less commonly, a P100 (16 GB, 3,584 CUDA cores, no Tensor Cores, slightly faster fp32 but no AMP speedup). We size the training run to fit on the smaller / slower of the two — the T4 — so we never have to redesign for an unlucky session-level allocation.

9.1 Memory budget

The dominant memory cost is the activations cached for backpropagation, not the parameters themselves. We compute the budget level by level for batch B = 16 (fp32; AMP halves these numbers):

Layer	Activation tensor shape	Forward MB (per sample)	With grad+optim (×3) [MB]
Encoder 1	(B, 32, 256, 256)	8.39	~25
Encoder 2	(B, 64, 128, 128)	4.19	~13
Encoder 3	(B, 128, 64, 64)	2.10	~6
Encoder 4	(B, 256, 32, 32)	1.05	~3
Bottleneck	(B, 512, 16, 16)	0.52	~2
Decoder 1–4	mirror of encoder	~15	~45
Output head	(B, 3, 256, 256)	0.79	~2
Skip concats	additive overhead	~5	~15

Forward activations total ≈ 110 MB. Adding the backward graph (×2–3 over forward) and the Adam optimiser state (parameters in fp32 + first moment + second moment ≈ 3× parameter count), total VRAM consumption is on the order of **1.5–2.5 GB**. Add cuDNN workspace (typically allocated 1–2 GB for convolution algorithms) and PyTorch's memory caching overhead, and the effective consumption is closer to **4–6 GB**. This leaves comfortable headroom under the T4's 16 GB ceiling — enough to switch to AMP off, double the batch, or absorb the extra cost of hyperbolic operations in Phase 3 without redesign.

9.2 Compute budget

Empirical throughput estimates (extrapolated from comparable U-Net workloads in the BasicSR / NAFNet codebases on T4):

- ≈ 4–6 iterations per second at batch 16, 256² input, mixed precision, on T4.
- Training set ≈ 1920 images × 8 patches/image / batch 16 ≈ 960 iterations per epoch.
- ≈ 3–4 minutes per epoch.
- 100 epochs ≈ 5–7 hours wall-clock — comfortably within Kaggle's 12-hour session limit.
- Validation pass (640 images, batch 16, full forward only) ≈ 1 minute, run every 5 epochs.

The numbers above are conservative; AMP on T4 Tensor Cores often delivers higher than the low estimate. Even at worst case the full run fits within a single Kaggle session, which removes the need to checkpoint-and-resume across sessions (a frequent source of subtle bugs).

9.3 Reproducibility

All random number generators are seeded explicitly (Python, NumPy, PyTorch CPU, PyTorch CUDA). cuDNN is set to deterministic mode for the validation pass and benchmark mode for training (the cuDNN benchmark mode introduces minor non-determinism in exchange for ~10% speedup, which is the standard restoration-literature compromise). Final reported numbers are produced from a deterministic evaluation run, so the headline metrics are exactly reproducible regardless of training-time non-determinism.

10. Evaluation protocol

10.1 PSNR — Peak Signal-to-Noise Ratio

PSNR is the de-facto standard image-restoration metric:

$$\mathbf{PSNR = 10 \cdot \log_{10}(MAX^2 / MSE)}$$

with MAX = 1 (images in [0, 1]) and MSE the mean squared error between the prediction and the clean ground truth. PSNR is interpretable as a logarithmic measure of reconstruction fidelity: each additional dB corresponds to a 26% reduction in MSE. In the denoising literature, a 0.2 dB improvement is publishable; 0.5 dB is notable; 1.0 dB is a major result.

We compute PSNR **after** clipping the prediction to [0, 1] and converting to 8-bit uint8 (multiply by 255, round, clip). This matches the convention used by every restoration benchmark we will compare against and avoids the subtle accounting differences that arise when PSNR is computed on float in-the-loop predictions.

10.2 SSIM — Structural Similarity

SSIM (Wang, Bovik, Sheikh & Simoncelli, IEEE TIP 2004) is a perceptual metric that evaluates whether the structural content of the prediction matches the ground truth, rather than just pixel-wise error. It is computed on local 11×11 windows and combines luminance, contrast, and structural-correlation terms into a single score in [-1, 1] (1 = perfect).

SSIM is complementary to PSNR: a model can have high PSNR but poor SSIM if it blurs out fine textures; conversely a model can have moderate PSNR but high SSIM if it preserves the structure of the image while making small per-pixel errors. Reporting both is standard.

We use the standard implementation in `torchmetrics`, which matches the original Wang et al. specification with default Gaussian window σ = 1.5.

10.3 Per-noise-level breakdown

Reporting a single PSNR averaged across all noise levels obscures the model's behaviour. We instead report:

- PSNR and SSIM at **light** noise (α = 0.5, σ = 3.0)
- PSNR and SSIM at **medium** noise (α = 1.0, σ = 5.0)
- PSNR and SSIM at **heavy** noise (α = 2.0, σ = 8.0)
- The **baseline PSNR** of the noisy input itself — so we can compute the improvement (dB) the denoiser provides at each level. This is more informative than the absolute PSNR; a 28 dB output is a great result if the input was 22 dB and a mediocre one if the input was 27 dB.

All metrics are computed on the held-out Test_Set (640 images, never seen during training or model selection).

10.4 Visual evaluation

Numerical metrics are necessary but not sufficient. We also save, every 10 epochs and at the end of training, a grid of **4-panel comparisons**: clean / noisy / denoised / residual (noisy − denoised), for a fixed set of validation images and a fixed seed. These let us catch failure modes that PSNR/SSIM hide: residual noise patterns, blurred vasculature, colour shifts, ring artifacts at tile boundaries.

The fixed validation images and fixed seed are critical: the visual comparison only tells us anything if the same image is used at every checkpoint, so improvements over time are visible directly.

10.5 What we will not measure (and why)

We deliberately exclude perceptual metrics that depend on a pre-trained network (LPIPS, PieAPP, DISTS). They are appropriate for natural-image restoration but introduce a domain mismatch on retinal images (the pre-trained VGG was never exposed to fundus photography, so its feature space may rank reconstructions in idiosyncratic ways). For the same reason we do not use FID or KID, which are designed for generative models, not regression.

Future work (Phase 4): a downstream-task evaluation — denoise the test set, then measure how a fixed retinal disease-classification model performs on the denoised images vs. the noisy ones. This is the gold-standard evaluation for medical denoising and is where the hyperbolic variant should pay its biggest dividends if the noise-robustness claim holds.

11. Adaptation deltas from the upstream codebase

This section enumerates every code change required to convert the upstream segmentation framework into a denoising framework. The intent is to make the diff auditable: a reviewer can verify each delta is justified and complete.

Component	Change	Justification (cross-reference)
Output head	Replace N-class softmax conv with linear 3-channel conv.	Section 5 — formal change of task from classification to
Forward wrapper	Subclass the U-Net; override forward() to return noisy [illegible] predict(noisy).	Section 5 — residual learning.
Loss function	Remove Dice + Focal/Tversky; install nn.L1Loss applied in the image domain.	Section 6 — L1 is the restoration-literature default.
Metrics	Replace Dice/IoU/Hausdorff with PSNR + SSIM from torchmetrics.	Section 10 — restoration is evaluated in dB and SSIM, n
Dataset class	Write RFMiDDataset reading from Kaggle mount; apply [illegible]	Section 7 — RFMiD is classification-labelled, so [illegible]
Augmentation pipeline	Strip colour jitter, geometric warps, blur, sharpening from upstream's augmentation stack; ke	Section 7.3 — [illegible]
Normalization	Switch BatchNorm to GroupNorm (or remove entirely).	Section 4.4 — BatchNorm interferes with the low-magn
Optimizer	Keep AdamW for Phase 2 (Euclidean); add Riemannian Adam [illegible]	Section 8.1 — standard, plus stay consistent with [illegible]
Logging	Replace Weights & Biases callback with tqdm [illegible]	Section 8 — [illegible] training/validation curves [illegible]
Train/val split	Use RFMiD's official Training_Set / Evaluation_Set / Test_Set folders.	Section 7.5 — honour the dataset's published splits.
Output channels	Set --classes 3 (RGB regression) rather than 2 (binary segmentation).	Sections 4 and 5.
Inference	Add full-image tiled inference with Hann-window blending; [illegible]	Section 7.2 — full-resolution images do not fit in memo

Of these 12 deltas, only three (the output head, the loss, and the dataset class) involve any non-trivial new code. The remaining nine are configuration edits, flag changes, or removals. This is exactly the small-surface-area adaptation that motivated choosing a published reference in the first place: we modify the absolute minimum required to repurpose the architecture for our task, leaving the validated parts intact.

12. The path to Phase 3 (hyperbolic variant)

By design, Phase 3 reduces to a single flag change in the upstream framework: `--model hyp` instead of `--model euc`. Mechanically, this swaps every encoder/decoder block from `torch.nn.Conv2d` / Linear / activation to the matching hypll modules (Möbius convolutions and Möbius activations operating in the Poincaré ball). Skip connections continue to carry Euclidean tensors and are bridged across the hyperbolic interior by the appropriate exp/log maps at the block boundaries. The Riemannian Adam optimiser is enabled automatically when a hyperbolic parameter is detected by hypll.

The only Phase 2 design decision that needs a Phase 3 reconsideration is mixed precision: the arctanh in the log map can overflow in fp16 as $\|y\| \to 1$. The standard mitigation (clamp $\|y\| \leq 1 - 10^{-5}$, force fp32 around exp/log/Möbius blocks) is built into hypll. We will verify on a 1-epoch sanity run that the hyperbolic AMP training is stable; if not, we disable AMP for Phase 3 and accept the ~2× slowdown.

Everything else — data, loss, optimiser type, metrics, evaluation protocol, the residual wrapper — is shared verbatim with Phase 2. The Phase 3 deliverable will therefore consist of two further notebook cells in the same Kaggle notebook (a hyperbolic training run + a side-by-side evaluation cell), not a separate codebase.

13. Risks and mitigations

Risk	Scenario	Mitigation
Upstream dependency breakage	hypll updates change the API and break our notebook.	Pin hypll==0.1.1 (the v
Kaggle session timeout	12-hour limit exceeded mid-training.	Budget 100 epochs at ≈ 4 min/epoch ⇒ ~7 hours. Checkpoi
Numerical instability (Phase 3)	arctanh blowup / NaN gradients in hyperbolic layers under AMP.	Disable AMP for the hyperbolic run if a sanity-check epoch is
Synthetic-real noise mismatch	Model trained on Poisson-Gaussian may not generalise to true clinical noise.	Acknowledge as a limitation; in Phase 4 a downstream class
Insufficient capacity	32-base U-Net underperforms because the task happens to need more parameters.	Channel width is a single config flag (<font face="DejaVu
Overfitting	Model memorises training set.	On-the-fly noise (Section 7.4) means every sample is unique
Comparison contamination	Phase 3 hyperbolic results differ for spurious reasons (different hyperparameters).	By construction we change only --model. All hyperparameter

14. Summary decisions log

All decisions in this document, consolidated for quick reference and committee Q&A.
Each row points to the section where the choice is defended.

Decision	Choice	Rationale
Base repo	swastishreya/Hyperbolic-U-Net (MIDL 2026)	§3 — only candidate satisfying all 5 selection criteria; match
Build vs. adapt	Adapt upstream, do not build from scratch	§2 — credibility, apples-to-apples comparison, code reuse, s
Backbone	U-Net (4-level encoder/decoder + skips)	§4.1, §4.2 — canonical dense-prediction architecture; recept
Channel widths	32 / 64 / 128 / 256 / 512	§4.3 — half of classic; fits T4 memory, denoising is structura
Normalization	GroupNorm (or no norm); not BatchNorm	§4.4 — BatchNorm removes the low-energy noise signal we
Activation	ReLU (unchanged from upstream)	§4.5 — gratuitous changes reduce comparability; alternative
Output head	1×1 Conv → 3 channels, linear	§5.3 — replaces upstream segmentation softmax; no activat
Forward semantics	Predict noise n̂; return y − n̂	§5.2 — residual learning, +0.3–1.0 dB PSNR free, easier opti
Loss	L1 (MAE) in image domain	§6.2 — Zhao et al. TCI 2017 result that L1 outperforms MSE
Patch size	256 × 256 random crops	§7.1 — receptive-field appropriate, memory-cheap, restorati
Full-image inference	Tile with Hann-window blending	§7.2 — full RFMiD resolution does not fit; tiling is standard at
Augmentations	HFlip + VFlip + Rot90 only	§7.3 — the dihedral group preserves noise statistics; colour/
Noise injection	Poisson-Gaussian on-the-fly; level uniform per iteration	§7.4 — fresh sample per iteration prevents memorisation; mixed levels yiel
Splits	Official RFMiD Training/Evaluation/Test	§7.5 — honour the dataset's published splits to avoid leakag
Optimizer	AdamW, lr = 1e-4, wd = 1e-4	§8.1 — decoupled weight decay; standard restoration defaul
LR schedule	Cosine annealing, per-iteration	§8.2 — no step-boundary hyperparameters; restoration-litera
Precision	AMP (fp16 forward / fp32 master)	§8.3 — ~2× speedup, ~½ memory, negligible PSNR cost on
Batch size	16 (fallback: 8 + 2-step grad accumulation)	§8.4, §9.1 — fits T4 comfortably; equivalent fallback if Phase
Epochs	100	§8.5 — extrapolates to ≈ 5–7 h on T4, within the 12 h sessio
Metrics	PSNR + SSIM, per noise level + delta vs. input	§10.1–10.3 — restoration standard; per-level breakdown give
Logging	tqdm + matplotlib (no W&B for submission)	§8 first summary, §11 deltas — simpler demo; W&B can be
Phase 3 transition	Single flag change (--model hyp)	§12 — by construction; only AMP needs revisiting for hyperb

15. References

- Mishra, S. S., van Spengler, M., Berkhout, E., & Mettes, P. (2026). *Hyperbolic U-Net for Robust Medical Image Segmentation.* Medical Imaging with Deep Learning (MIDL). OpenReview NxKaeTNMxR. Repository: `github.com/swastishreya/Hyperbolic-U-Net`.
- Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation.* MICCAI. arXiv:1505.04597.
- Zhang, K., Zuo, W., Chen, Y., Meng, D., & Zhang, L. (2017). *Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising* (DnCNN). IEEE TIP 26(7), 3142–3155.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition.* CVPR. arXiv:1512.03385. (Source of the general residual-learning principle.)
- Lim, B., Son, S., Kim, H., Nah, S., & Lee, K. M. (2017). *Enhanced Deep Residual Networks for Single Image Super-Resolution* (EDSR). CVPR Workshops. (Removed BatchNorm in restoration networks; L1 loss.)
- Ioffe, S. & Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.* ICML.
- Wu, Y. & He, K. (2018). *Group Normalization.* ECCV. arXiv:1803.08494.
- Zhao, H., Gallo, O., Frosio, I., & Kautz, J. (2017). *Loss Functions for Image Restoration with Neural Networks.* IEEE TCI 3(1), 47–57. (Foundational L1 vs MSE comparison.)
- Loshchilov, I. & Hutter, F. (2017). *SGDR: Stochastic Gradient Descent with Warm Restarts.* ICLR. (Cosine annealing.)
- Loshchilov, I. & Hutter, F. (2019). *Decoupled Weight Decay Regularization* (AdamW). ICLR.
- Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). *Image Quality Assessment: From Error Visibility to Structural Similarity* (SSIM). IEEE TIP 13(4), 600–612.
- Ungar, A. A. (2008). *Analytic Hyperbolic Geometry and Albert Einstein's Special Theory of Relativity.* World Scientific. (Gyrovector spaces / Möbius operations.)
- Ganea, O.-E., Bécigneul, G., & Hofmann, T. (2018). *Hyperbolic Neural Networks.* NeurIPS. arXiv:1805.09112.
- Nickel, M. & Kiela, D. (2017). *Poincaré Embeddings for Learning Hierarchical Representations.* NeurIPS. arXiv:1705.08039.
- Shimizu, R., Mukuta, Y., & Harada, T. (2021). *Hyperbolic Neural Networks++.* ICLR. arXiv:2006.08210.
- Bdeir, A., Falk, K., & Landwehr, N. (2024). *Fully Hyperbolic Convolutional Neural Networks for Computer Vision* (HCNN). ICLR. arXiv:2303.15919.
- van Spengler, M., Berkhout, E., & Mettes, P. (2023). *Poincaré ResNet.* ICCV. arXiv:2303.14027. (hypll's foundation.)
- Atigh, M. G., Schoep, J., Acar, E., van Noord, N., & Mettes, P. (2022). *Hyperbolic Image Segmentation.* CVPR. arXiv:2203.05898.
- Pachade, S., et al. (2021). *Retinal Fundus Multi-disease Image Dataset (RFMiD): A Dataset for Multi-Disease Detection Research.* Data 6(2), 14.
- Fu, H., et al. (2020). *Modeling and Enhancing Low-Quality Retinal Fundus Images.* IEEE TMI 40(3), 996–1006. Repo: `github.com/HzFu/EyeQ_Enhancement`. (Reference for retinal-specific preprocessing.)
- Chen, L., Chu, X., Zhang, X., & Sun, J. (2022). *Simple Baselines for Image Restoration* (NAFNet). ECCV. arXiv:2204.04676. (Modern restoration baseline; precedent for many of the training-recipe choices.)
- Liang, J., et al. (2021). *SwinIR: Image Restoration Using Swin Transformer.* ICCV Workshops. arXiv:2108.10257.

• Zamir, S. W., et al. (2022). *Restormer: Efficient Transformer for High-Resolution Image Restoration.* CVPR. arXiv:2111.09881.